

# Dan Boneh 密码学专题：CBC 的运行机制

CBC 的全称是 Cipher Block Chaining，即密文分组链接模式。理解它的关键是：每个明文分组在加密前，先与前一个密文分组异或，再交给分组密码加密。

## 1. 为什么需要 CBC

AES 这样的分组密码一次只能处理固定长度的数据。例如，AES 每次处理 128 位：

$$E_k : \{0, 1\}^{128} \rightarrow \{0, 1\}^{128}.$$

长消息需要先划分成多个分组：

$$M = M_1 \parallel M_2 \parallel \cdots \parallel M_n.$$

如果直接分别加密每个分组：

$$C_i = E_k(M_i),$$

就是 ECB 模式。它存在一个严重问题：

$$M_i = M_j \implies C_i = C_j.$$

相同明文块会产生相同密文块，从而泄露消息中的重复结构。CBC 通过让每个分组的加密依赖前一个密文分组，隐藏这种重复模式。

## 2. CBC 加密过程

首先生成一个与分组同样长的初始化向量 IV，并规定：

$$C_0 = IV.$$

然后依次计算：

$$C_i = E_k(M_i \oplus C_{i-1}).$$

因此：

$$C_1 = E_k(M_1 \oplus IV),$$

$$C_2 = E_k(M_2 \oplus C_1),$$

$$C_3 = E_k(M_3 \oplus C_2).$$

可以记成：

第一个分组：M1 XOR IV → 分组密码加密 → C1  
第二个分组：M2 XOR C1 → 分组密码加密 → C2  
第三个分组：M3 XOR C2 → 分组密码加密 → C3

最终发送的内容通常是：

$$IV \parallel C_1 \parallel C_2 \parallel \cdots \parallel C_n.$$

IV 一般不需要保密，可以和密文一起发送，但必须满足正确的生成和完整性要求。

---

### 3. 第一个分组为什么需要 IV

从第二个明文分组开始，可以和前一个密文分组异或。但第一个明文分组前面没有密文，因此使用 IV：

$$C_1 = E_k(M_1 \oplus IV).$$

如果没有随机变化的 IV，那么两条消息只要第一个明文分组相同，第一个密文分组也会相同。

假设两次加密使用不同的 IV：

$$IV \neq IV'.$$

即使两条消息的第一个分组相同：

$$M_1 = M'_1,$$

通常也有：

$$M_1 \oplus IV \neq M'_1 \oplus IV'.$$

因此，产生的第一个密文分组通常不同。

---

### 4. 一个简化例子

为了便于观察，假设每个分组只有 4 位。实际 AES 的分组为 128 位。

设：

```
M1 = 1010
M2 = 1100
IV = 0110
```

先处理第一个分组：

```
M1 XOR IV
= 1010 XOR 0110
= 1100
```

然后把 1100 交给分组密码。假设：

$$C_1 = E_k(1100) = 0011$$

再处理第二个分组：

$$\begin{aligned} &M_2 \text{ XOR } C_1 \\ &= 1100 \text{ XOR } 0011 \\ &= 1111 \end{aligned}$$

随后计算：

$$C_2 = E_k(1111)$$

可以看到， $C_2$  不只取决于  $M_2$ ，还通过  $C_1$  间接取决于前面的消息内容。这就是 Chaining，即“链接”的含义。

## 5. CBC 解密过程

加密公式是：

$$C_i = E_k(M_i \oplus C_{i-1}).$$

先使用分组密码的逆运算：

$$D_k(C_i) = M_i \oplus C_{i-1}.$$

然后再异或前一个密文分组：

$$M_i = D_k(C_i) \oplus C_{i-1}.$$

因此：

$$M_1 = D_k(C_1) \oplus IV,$$

$$M_2 = D_k(C_2) \oplus C_1,$$

$$M_3 = D_k(C_3) \oplus C_2.$$

图示为：

```
C1 → 分组密码解密 → XOR IV → M1
C2 → 分组密码解密 → XOR C1 → M2
C3 → 分组密码解密 → XOR C2 → M3
```

解密能够恢复明文，是因为异或同一个值两次会抵消：

$$(x \oplus y) \oplus y = x.$$

以第  $i$  个分组为例：

$$D_k(C_i) \oplus C_{i-1} = (M_i \oplus C_{i-1}) \oplus C_{i-1} = M_i.$$

## 6. 为什么相同明文块通常不会产生相同密文块

假设同一条消息中：

$$M_2 = M_5.$$

CBC 实际计算的是：

$$C_2 = E_k(M_2 \oplus C_1),$$

$$C_5 = E_k(M_5 \oplus C_4).$$

虽然  $M_2=M_5$ ，但通常  $C_1 \neq C_4$ ，所以送入分组密码的两个输入不同，最终密文通常也不同。

需要注意，CBC 隐藏的是单个消息中的重复分组模式。若错误地在同一密钥下重复使用 IV，相同的消息前缀仍可能产生相同的密文前缀。

## 7. CBC 的填充

CBC 要求每个明文分组都具有完整的分组长度。如果消息长度不是分组长度的整数倍，就需要填充。

常见方案是 PKCS#7。填充的每个字节都等于填充字节数。

例如，分组大小为 8 字节，最后一个分组还缺 3 字节，则补上：

03 03 03

如果消息刚好是完整分组，仍然要增加一个完整的填充分组。若分组大小为 8 字节，则补上：

08 08 08 08 08 08 08 08

为什么完整分组还要填充？假如不额外填充，解密方看到明文末尾的 01 或 02 02 时，就无法判断这些字节究竟是原消息，还是填充内容。

AES 的分组大小为 16 字节，所以 PKCS#7 填充长度会在 1 到 16 字节之间。

## 8. CBC 的串行与并行性质

### 8.1 加密必须串行

计算  $C_2$  前必须先得到  $C_1$ ：

$$C_2 = E_k(M_2 \oplus C_1).$$

计算  $C_3$  又必须先得到  $C_2$ 。因此，CBC 加密天然具有前后依赖，不能直接并行计算所有密文分组。

## 8.2 解密可以并行

当完整密文已经收到时，每个分组都可以分别计算：

$$M_i = D_k(C_i) \oplus C_{i-1}.$$

计算  $M_i$  所需的  $C_i$  与  $C_{i-1}$  都已经存在，所以可以并行执行各个  $D_k(C_i)$ 。

| 操作     | 能否并行 | 原因          |
|--------|------|-------------|
| CBC 加密 | 通常不能 | 当前密文依赖前一个密文 |
| CBC 解密 | 可以   | 所需的相邻密文都已知  |

## 9. 密文错误如何传播

假设传输过程中，密文分组  $C_i$  的某一位发生翻转。

### 9.1 对 $M_i$ 的影响

$$M_i = D_k(C_i) \oplus C_{i-1}.$$

因为分组密码输入  $C_i$  被改变， $D_k(C_i)$  通常会整体发生不可预测的变化。因此， $M_i$  整个分组通常都会损坏。

### 9.2 对 $M_{i+1}$ 的影响

$$M_{i+1} = D_k(C_{i+1}) \oplus C_i.$$

$C_i$  在这里直接参与异或，因此  $C_i$  的哪一位翻转， $M_{i+1}$  的对应位也会翻转。

### 9.3 对后续分组的影响

从  $M_{i+2}$  开始，计算不再使用  $C_i$ ，所以如果没有其他错误，后面的明文分组会恢复正常。

可以记成：

| 被修改的密文 | 对应明文的影响             |
|--------|---------------------|
| $C_i$  | $M_i$ 整块不可预测地损坏     |
| $C_i$  | $M_{i+1}$ 的对应位翻转    |
| $C_i$  | $M_{i+2}$ 及之后通常不受影响 |

## 10. CBC 的可篡改性

CBC 只提供保密性，不自动提供完整性。

由解密公式：

$$M_i = D_k(C_i) \oplus C_{i-1}$$

可知，如果攻击者把前一个密文分组修改为：

$$C'_{i-1} = C_{i-1} \oplus \Delta,$$

那么第  $i$  个明文分组会变成：

$$\begin{aligned} M'_i &= D_k(C_i) \oplus C'_{i-1} \\ &= D_k(C_i) \oplus C_{i-1} \oplus \Delta \\ &= M_i \oplus \Delta. \end{aligned}$$

这意味着攻击者虽然不知道密钥，也可能有控制地翻转下一明文分组中的某些比特。

与此同时，被修改的  $C_{i-1}$  自身对应的明文分组  $M_{i-1}$  会不可预测地损坏。但在某些消息格式中，攻击者仍可能利用这一性质实施比特翻转攻击。

因此：

能够成功解密，不代表消息没有被篡改。

传统系统需要把 CBC 和 MAC 按照正确顺序组合。现代系统通常优先使用 AES-GCM、ChaCha20-Poly1305 等认证加密方案，同时提供保密性和完整性。

## 11. IV 的使用要求

CBC 中的 IV：

- 不需要保密；
- 在标准 CBC 保密性要求下应当不可预测；
- 在同一个密钥下不应重复使用；
- 必须受到完整性保护；
- 长度必须等于分组密码的分组长度。

### 11.1 重复 IV 的问题

如果同一密钥下重复使用相同 IV，并且两条消息的第一个分组相同：

$$M_1 = M'_1,$$

那么：

$$\begin{aligned} C_1 &= E_k(M_1 \oplus IV) \\ &= E_k(M'_1 \oplus IV) \\ &= C'_1. \end{aligned}$$

攻击者因此可以发现两条消息拥有相同的开头。若多个前缀分组都相同，相应的密文前缀也会相同。

### 11.2 修改 IV 的问题

第一个明文分组为：

$$M_1 = D_k(C_1) \oplus IV.$$

若攻击者把 IV 改成：

$$IV' = IV \oplus \Delta,$$

解密后的第一个分组会变成：

$$M'_1 = M_1 \oplus \Delta.$$

因此，IV 虽然可以公开，却不能允许攻击者在没有被检测的情况下随意修改。

---

## 12. CBC 与 ECB、CTR 的简单比较

| 特性              | ECB | CBC       | CTR                |
|-----------------|-----|-----------|--------------------|
| 是否隐藏重复明文块       | 否   | 是         | 是                  |
| 是否需要 IV 或 nonce | 否   | 需要不可预测 IV | 需要唯一 nonce/counter |
| 是否需要填充          | 需要  | 需要        | 不需要                |
| 加密能否并行          | 可以  | 不能直接并行    | 可以                 |
| 解密能否并行          | 可以  | 可以        | 可以                 |
| 是否自动提供完整性       | 否   | 否         | 否                  |

无论 CBC 还是 CTR，单独使用都不能防止主动篡改。若没有额外认证，应当视为不完整的加密方案。

---

## 13. 常见误区

误区 1：CBC 就是把消息分组后分别加密

错误。分别独立加密是 ECB。CBC 在加密当前明文分组前，还要异或前一个密文分组。

误区 2：IV 是另一把秘密密钥

错误。IV 通常可以公开，并与密文一起发送。它的作用是让相同消息在不同加密中产生不同密文。

误区 3：IV 只要不公开就可以重复使用

错误。IV 是否公开不是核心问题；CBC 要求 IV 在相应安全模型下不可预测，并避免在同一密钥下重复。

误区 4：CBC 解密也必须完全串行

错误。完整密文到达后，每个  $D_k(C_i)$  可以并行计算，再分别异或  $C_{i-1}$ 。

误区 5：加密后的数据无法被攻击者有意义地修改

错误。CBC 具有可篡改性，攻击者可以通过修改前一密文块控制后一明文块的比特翻转。

误区 6：随机 IV 可以让 CBC 同时提供完整性

错误。随机 IV 主要用于语义安全，不能替代 MAC 或认证加密。

## 误区 7：只要填充格式正确，密文就一定可信

错误。填充只解决分组对齐。若系统把填充是否正确信息泄露给攻击者，还可能形成 Padding Oracle Attack，即填充预言机攻击。

---

## 14. 必记公式

令：

$$C_0 = IV.$$

CBC 加密：

$$C_i = E_k(M_i \oplus C_{i-1}).$$

CBC 解密：

$$M_i = D_k(C_i) \oplus C_{i-1}.$$

比特翻转关系：

$$C'_{i-1} = C_{i-1} \oplus \Delta \implies M'_i = M_i \oplus \Delta.$$

---

## 15. 自测题

题目

1. CBC 中，第一个明文分组与什么值异或？
2. 写出 CBC 的加密和解密公式。
3. 为什么相同的明文分组出现在不同位置时，通常不会产生相同密文？
4. 为什么 CBC 加密不能直接并行？
5. 为什么 CBC 解密可以并行？
6. 修改  $C_i$  的一个比特，会如何影响  $M_i$  与  $M_{i+1}$ ？
7. IV 是否需要保密？它需要满足哪些安全要求？
8. 为什么消息长度刚好是分组长度的整数倍时，PKCS#7 仍要增加一整个填充分组？
9. CBC 是否能够检测攻击者对密文的篡改？
10. 现代系统为什么通常优先使用 AES-GCM 等认证加密方案？

参考答案

1. 与 IV 异或，可以把 IV 看作  $C_0$ 。
2. 加密为  $C_i = E_k(M_i \oplus C_{i-1})$ ，解密为  $M_i = D_k(C_i) \oplus C_{i-1}$ 。
3. 因为加密前还会异或不同的前置密文分组，送入分组密码的输入通常不同。
4. 计算  $C_i$  必须先知道  $C_{i-1}$ ，存在前后依赖。
5. 完整密文收到后，计算每个  $M_i$  所需的  $C_i$  和  $C_{i-1}$  都已经存在。
6.  $M_i$  通常整块不可预测地损坏； $M_{i+1}$  的对应位会翻转。
7. IV 通常不需要保密，但应不可预测、不在同一密钥下重复，并受到完整性保护。
8. 这样解密方才能无歧义地判断并删除填充，避免把原消息结尾误认为填充。
9. 不能。CBC 本身具有可篡改性，不提供消息认证。



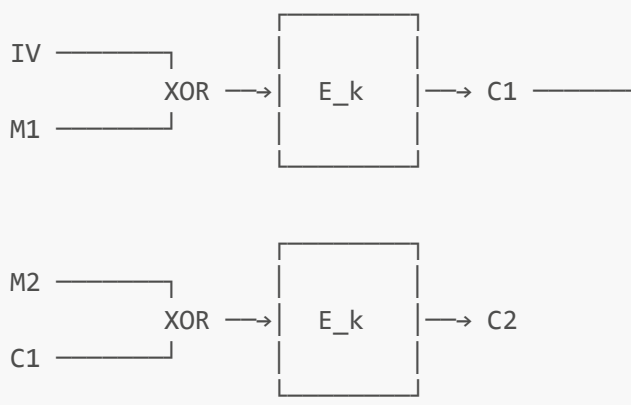
10. 认证加密能够在标准方案中同时提供保密性、完整性和来源认证，减少组合错误和填充预言机等风险。

## 16. 一分钟复习

1. CBC 不是独立加密各分组，而是让当前分组与前一密文分组相链接。
2. 第一个分组没有前置密文，因此使用 IV，并令  $C_0=IV$ 。
3. 加密时先异或、再加密： $C_i=E_k(M_i \oplus C_{i-1})$ 。
4. 解密时先解密、再异或： $M_i=D_k(C_i) \oplus C_{i-1}$ 。
5. CBC 加密必须串行，收到完整密文后解密可以并行。
6. CBC 需要填充，常见方法是 PKCS#7。
7. IV 可以公开，但应不可预测、不得在同一密钥下重复，并应受到完整性保护。
8. 修改一个密文块会破坏当前明文块，并翻转下一明文块的对应比特。
9. CBC 只提供保密性，不提供完整性，不能单独抵抗主动攻击者。
10. 新系统通常优先使用 AES-GCM 或 ChaCha20-Poly1305 等认证加密方案。

## 17. 最简记忆图

CBC 加密：



口诀：

加密先异或，再加密；  
解密先解密，再异或；  
异或前一个密文块；  
第一个块使用 IV。